

Porting ThunderKittens from CUDA to SYCL for Intel GPU: Process, Challenges, and Lessons Learned

Yehong Jiang*
Intel Corporation
Santa Clara, CA, USA
Department of Electrical Engineering
and Computer Science
Massachusetts Institute of Technology
CAMBRIDGE, Massachusetts, USA
yehong.jiang@intel.com

Shen Chen
Intel Corporation
Shanghai, Massachusetts, China
sheng.s.chen@intel.com

Fangwen Fu
Intel Corporation
Santa Clara, CA, USA
fangwen.fu@intel.com

Vincent Lu
Intel Corporation
Santa Clara, CA, USA
vincent.lu@intel.com

Yen-Kuang Chen
Intel Corporation
Santa Clara, CA, USA
y.k.chen@intel.com

Hong Wang
Intel Corporation
Santa Clara, CA, USA
hong.wang@intel.com

Xinmin Tian
Intel Corporation
Santa Clara, CA, USA
xinmin.tian@intel.com

Abstract

This paper presents our experience porting ThunderKittens, a CUDA-based GPU programming framework for deep learning kernels, to SYCL for Intel GPUs. We follow a three-stage process: automated conversion, manual adaptation, and architecture-specific optimization. The main task is translating ThunderKittens' C++ template library and replacing PTX-based implementations with alternatives for Intel GPUs. This required changes to both register layout and memory movement, replacing CUDA's shared-memory-centric execution path with register layouts better suited to Intel GPUs and L1-prefetch-based latency hiding. Our results show where current CUDA-to-SYCL translation tools help and where manual porting is still required. Although full performance portability remains difficult across architectures, the same high-level abstractions can be preserved across platforms: our SYCL port comes within 5% of Intel's hand-optimized oneDNN GEMM performance and exceeds it on the fused kernels we evaluated. These results show what is required to port framework-level GPU abstractions across architectures.

CCS Concepts

• **Computing methodologies** → **Parallel programming languages**.

*Work performed while the author was an intern at Intel Corporation. The author is also a student in the Department of Electrical Engineering and Computer Science at the Massachusetts Institute of Technology.



This work is licensed under a Creative Commons Attribution 4.0 International License. *IWOCL '26, Heilbronn, Germany*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2499-2/26/05
<https://doi.org/10.1145/3811257.3811269>

Keywords

CUDA to SYCL porting, Cross-platform GPU programming, Performance portability, ThunderKittens, Intel GPU optimization, SYCLomatic

ACM Reference Format:

Yehong Jiang, Shen Chen, Fangwen Fu, Vincent Lu, Yen-Kuang Chen, Hong Wang, and Xinmin Tian. 2026. Porting ThunderKittens from CUDA to SYCL for Intel GPU: Process, Challenges, and Lessons Learned. In *International Workshop on OpenCL and SYCL (IWOCL '26)*, May 06–08, 2026, Heilbronn, Germany. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3811257.3811269>

1 Introduction

While programming models like SYCL [6] promise cross-platform portability through open standards, the practical challenges of migrating existing CUDA [8] codebases, especially complex, performance critical frameworks, are not well understood.

ThunderKittens (TK) [10] is a strong case study for cross-platform porting. As a modern C++ library for AI kernel development, it provides hardware-conscious abstractions that deliver both programming convenience and high performance on NVIDIA GPUs. The framework uses advanced CUDA features including Tensor Cores, asynchronous memory operations, warp-level primitives, and extensive inline PTX [9] assembly, making it typical of the GPU programming patterns found in high-performance machine learning systems.

TK organizes GPU computation around *tiles*, small matrix fragments that serve as the fundamental unit of data and computation. The framework defines tile types at three levels of the memory hierarchy: register tiles (rt) held in thread registers for arithmetic, shared tiles (st) residing in shared memory for cooperative data staging, and global tiles for bulk transfers. Kernels are composed by

invoking tile-level operations (e.g., load, store, mma, copy) that the C++ template library lowers to architecture-specific instructions, allowing developers to write concise, hardware-agnostic kernel logic while the library handles layout, synchronization, and data movement details. This tight coupling to vendor-specific features makes TK a useful case for studying the limits of cross-platform porting; concurrent work on HipKittens [1] reports similar challenges porting TK to AMD GPUs.

We ask whether complex, performance-critical GPU frameworks can be migrated across vendor ecosystems while maintaining both correctness and performance. The contributions of this work include:

- (1) A three-stage methodology for CUDA-to-SYCL porting that combines automation with manual expertise;
- (2) Detailed analysis of SYCLomatic’s capabilities and limitations when applied to complex frameworks;
- (3) Practical solutions and mapping strategies for translating PTX assembly instructions; and
- (4) Lessons learned and best practices for cross-platform GPU programming efforts.

2 Three-Stage Porting Methodology

As shown in Figure 1, our porting approach follows three stages that balance automated translation with the manual work required for complex GPU code. We present this structure because it matches how the port had to be completed in practice.

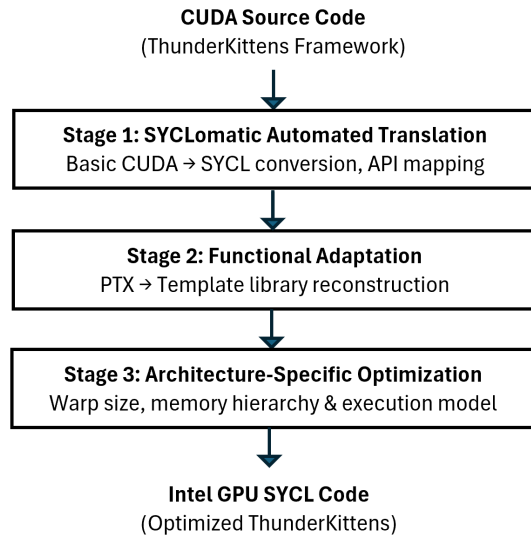


Figure 1: CUDA to SYCL porting workflow. The process proceeds through three stages, with iterative refinement between Stages 2 and 3 as architecture-specific optimizations may require revisiting functional adaptations.

2.1 Stage 1: Automated Conversion with SYCLomatic

We initiated the porting process using Intel’s SYCLomatic tool [2], an open-source command-line utility that provides automated translation from CUDA to SYCL. The conversion process involves a straightforward command-line invocation that processes the entire TK source tree.

SYCLomatic Strengths: The tool handles standard CUDA constructs well: basic kernel launches transform cleanly and predictably. SYCLomatic effectively handles memory management by migrating `cudaMalloc` to `sycl::malloc_device` and `cudaMallocManaged` to `sycl::malloc_shared` through Unified Shared Memory (USM), preserving the pointer-based access patterns TK relies on for performance and avoiding the overhead of SYCL’s buffer and accessor system.

Critical Limitations: The main obstacle was TK’s inline PTX assembly, which SYCLomatic comments out rather than translating. Our analysis identified 217 PTX assembly instructions across 19 unique instruction families. The largest groups were Hopper-exclusive [7] instructions such as WGMMMA and asynchronous primitives such as TMA bulk copies and `mbarrier`. As a result, the Stage 1 output was a useful starting point but not yet a functional port.

Tool Assessment: Despite these limitations, SYCLomatic still helps by handling boilerplate conversion automatically and establishing the basic SYCL framework structure. However, automated porting is a one-time effort that does not maintain synchronization with an evolving codebase. For complex libraries like TK, it is a starting point; framework functionality and performance still require manual reconstruction and architecture-specific optimization.

2.2 Stage 2: Functional Adaptation

The second stage restores functionality lost during automated translation by reconstructing TK’s PTX-backed template library in SYCL. The goal at this stage is correctness rather than performance.

PTX Assembly Translation Strategy: We first separated PTX usage into three groups. Hopper-exclusive instructions such as WGMMMA were bypassed because TK already provides equivalent non-Hopper code paths using standard `mma.sync` instructions. Asynchronous primitives such as TMA and `mbarrier` were also deferred because they require Intel-side support that is not yet available. For the remaining portable subset of 69 PTX instructions across seven functional categories—memory I/O (20), matrix operations (10), data type conversion (9), register shuffles (8), warp-level synchronization (4), predicate and control flow (4), and special-function math (14)—we used **SYCLomatic’s User-Defined Migration Rules**, YAML-based configuration files that specify custom source-to-source mappings. These rules replaced PTX-wrapping API calls with C helper functions that preserve semantics. For example, `ld/st.shared` and `ld/st.global` map to portable C++ loads and stores, while `mma.sync` maps to a helper that emulates MMA through scalar multiply-add loops over shuffled fragments. This made the code functional but not performant.

Execution Model Adaptation: Adapting from CUDA’s warp-based execution model to SYCL’s subgroup abstraction presented both conceptual and practical challenges. CUDA’s SIMT model assumes 32-thread warps (`WARP_SIZE = 32`) executing in lockstep,

while SYCL provides similar functionality through subgroups with different size assumptions and more explicit control. During this process, we identified and corrected several hardcoded 32-thread assumptions throughout the codebase to use proper parameterized values. Through systematic functional debugging, we verified that both 16- and 32-wide subgroup configurations produced correct results, deferring the selection of the performance-optimal size to Stage 3.

2.3 Stage 3: Architecture-Specific Optimization

This stage took most of the porting effort because it required understanding TK’s internal architecture, the NVIDIA-specific assumptions built into it, and the corresponding Intel GPU design choices. We approached this stage iteratively, cycling between modifying, redefining, and implementing TK’s template library alongside kernel implementation and optimization. Some structural changes required returning to Stage 2 to restore correctness when optimizations broke helper-based functionality.

Subgroup Size and Intrinsic Replacement: The first step in Stage 3 was selecting the Intel-specific execution model. CUDA’s SIMT model assumes 32-thread warps, whereas Intel tensor operations are best served by 16-wide subgroups. After validating both configurations functionally in Stage 2, we selected 16-wide subgroups to match the DPAS (Dot Product Accumulate Systolic) execution width. Only then could we replace the Stage 2 C helper functions with architecture-appropriate SPIR-V builtins and SYCL extensions that map directly to Intel GPU hardware units.

Register Tile Optimization: We increased register tile dimensions from CUDA’s optimal 16×16 configurations to 32×16 or 32×32 for Intel GPUs. While the 16×16 configuration has decent performance, this change achieves maximal performance with 64-byte cache line aligned fetches, improving data fetch efficiency and better utilizing Intel’s large L1 cache capacity (compared to NVIDIA’s smaller shared memory).

Register Vector and Tile Layout: NVIDIA utilizes a warp-level 8×8 tile abstraction for Tensor Core operations, supported by dedicated PTX instructions. This allows a 32-thread warp to efficiently load, transpose, and map matrix fragments into registers for high-throughput Matrix Multiply-Accumulate (MMA) execution. Specifically, TK uses PTX instructions such as `movmatrix.sync.aligned.m8n8.trans.b16` introduced in Hopper generation GPUs to perform register-to-register transpositions. In contrast, Intel GPUs use a more linear SIMD tile structure optimized for their DPAS (Dot Product Accumulate Systolic) architecture. We redesigned both vector and tile layout structures within TK’s template library, transitioning from NVIDIA’s warp-centric subtile approach to Intel’s subgroup-based linear SIMD organization. While TK’s design philosophy of abstracting layout details from kernel code meant that most kernels remained unchanged or required only minor modifications, the underlying template implementations required complete restructuring.

Memory Hierarchy Adaptations: NVIDIA and Intel GPUs favor different tensor data paths. NVIDIA GPUs are optimized for shared memory-centric workflows, using asynchronous global-to-shared memory load and store operations to stage data close to compute units before feeding it to Tensor Cores. In contrast,

Intel GPUs are optimized for L1 cache-centric data paths, using L1 prefetch mechanisms, cooperative cache prefetch strategies, and specialized 2D block load and store operations. Concretely, in the CUDA version of a GEMM main loop, each iteration issues an asynchronous copy of the next tile from global memory into shared memory (`load_async`), waits on the transfer via a barrier, and then feeds the shared tile to the matrix multiply-accumulate operation. In the Intel GPU version, we replace this pattern with a prefetch of the next tile into L1 cache (`prefetch_load`) issued several iterations ahead, followed by a 2D block load directly from global memory into registers when the data is needed; the hardware L1 cache absorbs the latency, eliminating the explicit shared-memory staging step entirely. We introduced new TK APIs such as `prefetch_load` to express this pattern, necessitating algorithmic changes in kernels that restructure the software pipeline around prefetch distance rather than shared-memory buffer management. These adaptations changed both TK’s API surface and how it moves data through the memory hierarchy.

Table 1 summarizes the main CUDA/PTX-to-Intel mappings that emerged from Stage 3. These are not always literal instruction-for-instruction replacements; in some cases, such as `cp.async`, the Intel implementation uses a higher-level abstraction that provides a comparable capability through a different execution strategy.

Table 1: Representative CUDA/PTX-to-Intel mappings used in the final TK port.

CUDA/PTX	Intel-side Mapping	Note
<code>ld.st.global</code>	<code>2d_block_read/write</code>	SPIR-V; LSC 2D
<code>cp.async</code>	<code>prefetch_load</code>	L1\$ prefetch
<code>mma.sync</code>	<code>matrix_multiply_accumulate</code>	XXM via DPAS
<code>shfl.sync</code>	<code>sub_group::shuffle</code>	SYCL std
<code>vote.sync</code>	<code>sub_group::all_of/any_of</code>	SYCL std
<code>bar.sync</code>	<code>group_barrier</code>	SYCL std

3 Results and Performance Evaluation

To validate our porting methodology, we evaluated Sycl-TK (our Sycl port of TK) against established baselines including oneDNN [3] and Sycl-TLA [5] on Intel Data Center GPU Max 1550 [4], covering both proprietary optimized libraries and open-source template frameworks. Reported data are the geometric mean of runs across diverse problem sizes (>10 configurations) representative of real-world ML workloads, with GEMM evaluations ranging from $1k \times 1k \times 1k$ to $16k \times 16k \times 16k$ dimensions including non-power-of-2 sizes.

As shown in Figure 2, Sycl-TK is competitive across representative ML workloads:

- **BF16 GEMM (BGEMM):** oneDNN outperforms Sycl-TK by 5%, while Sycl-TLA performs comparably (2% slower than Sycl-TK). On large configurations Sycl-TK sustains 177–213 TFlop/s BF16, matching or exceeding oneDNN on 4 of 12 shapes tested;
- **Add+LayerNorm:** Sycl-TK outperforms oneDNN by 17% due to kernel fusion advantages that eliminate intermediate memory traffic;

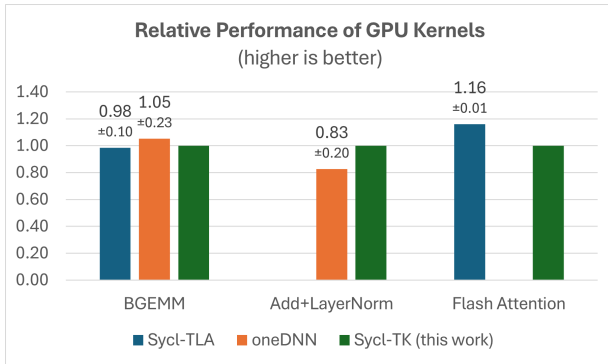


Figure 2: Performance of Sycl-TK, Sycl-TLA, and oneDNN on Intel Data Center GPU Max 1550, normalized to Sycl-TK. oneDNN does not provide a fused Flash Attention kernel.

- **Flash Attention:** Sycl-TLA currently outperforms Sycl-TK by 16%; optimization work continues to close this gap through improved memory access patterns and tile scheduling.

To illustrate the impact of each optimization stage on a representative BF16 GEMM, the baseline without prefetching achieves 78 TFlop/s (47% of oneDNN’s 168 TFlop/s). Adding per-subgroup L1 prefetch raises this to 121 TFlop/s (72%), enlarging tiles from 16×16 to 32×32 raises it to 144 TFlop/s (86%), and collaborative prefetching, where subgroups divide prefetch responsibilities to eliminate redundant memory operations, achieves 153 TFlop/s (91%). We use oneDNN rather than the theoretical peak as the practical ceiling because the Max 1550, like recent high-end GPUs from all vendors, is power-bound and sustains a reduced frequency under sustained matrix-engine workloads.

These results show that much of the original TK programming model can be preserved while achieving GEMM performance close to oneDNN and better performance on the fused kernels we evaluated.

4 Lessons Learned

4.1 Tooling and Methodology

SYCLomatic handled routine CUDA constructs reliably but could not translate PTX assembly or architecture-specific code; for complex libraries like TK, automated translation is a starting point, not a complete solution. YAML-based User-Defined Migration Rules let developers declare custom type and API mappings that SYCLomatic applies during translation, reducing repetitive manual edits. Separating correctness (Stages 1–2) from optimization (Stage 3) was essential: trying to optimize before the port compiles and runs correctly compounds debugging effort.

4.2 Abstraction and Portability

The most significant obstacles were architecture-specific rather than language-level. SYCL provides portable primitives for universal GPU concepts (subgroups, barriers, local memory) but does not cover architecture-specific features like asynchronous bulk data movement (cf. CUDA’s TMA), hardware-managed barriers, or warp-specialized pipeline scheduling; each vendor must supply these

through extensions or libraries. Identical TK kernels will not perform optimally on every GPU, but structural portability is practical: extending TK’s template-parameterized API with `#ifdef`-selected backends required adding roughly 12k lines of SYCL-specific code while sharing the algorithmic core, and optimal kernels on both NVIDIA and Intel share the same tile and vector abstractions and API surface.

4.3 Long-term Maintenance

Keeping the SYCL port in sync with upstream CUDA changes is an ongoing cost, since new features are not inherited automatically. Extending Sycl-TK to NVIDIA and AMD would still require vendor-specific intrinsics (e.g., `mma.sync`, MFMA), so architecture-specific branches remain necessary regardless of the language. When target platforms are few and upstream changes are frequent, maintaining a separate port is often cheaper than building a unified framework; as the number of targets grows, the framework approach amortizes per-platform engineering cost.

5 Conclusion and Future Work

Our experience suggests that cross-platform performance comes from keeping the abstraction portable while letting the implementation remain platform-specific. In our case, this approach comes within 5% of Intel’s hand-optimized oneDNN library for BGEMM and exceeds oneDNN on the fused kernels we evaluated.

Future work should extend this approach to additional GPU vendors and develop more sophisticated automated translation tools. As GPU architectures diverge further, reusable porting methodologies will be needed to keep cross-platform development practical.

Acknowledgments

We thank Simran Arora and William Hu from the ThunderKittens team at Stanford University for their valuable discussions supporting this work.

References

- [1] William Hu, Drew Wadsworth, Sean Siddens, Stanley Winata, Daniel Y. Fu, Ryann Swann, Muhammad Osama, Christopher Ré, and Simran Arora. 2025. HipKittens: Fast and Furious AMD Kernels. arXiv:2511.08083 [cs.LG] doi:10.48550/arXiv.2511.08083
- [2] Intel Corporation. 2022. SYCLomatic: A New CUDA-to-SYCL Code Migration Tool. Software tool. <https://www.intel.com/content/www/us/en/developer/tools/oneapi/dpc-compatibility-tool.html>
- [3] Intel Corporation. 2023. Intel oneAPI Deep Neural Network Library (oneDNN) Developer Guide. Intel Corporation, Santa Clara, CA, USA. <https://www.intel.com/content/www/us/en/developer/tools/oneapi/onednn.html>
- [4] Intel Corporation. 2023. Intel® Data Center GPU Max Series. Product Overview. Intel Corporation, Santa Clara, CA, USA. <https://www.intel.com/content/www/us/en/products/details/discrete-gpus/data-center-gpu/max-series.html>
- [5] Intel Corporation. 2024. SYCL Template Library for Accelerated Computing (SYCL-TLA). GitHub repository. <https://github.com/intel/sycl-tla>
- [6] Khronos Group. 2021. SYCL 2020 Specification. Standard. Khronos Group, Inc., Beaverton, OR, USA. <https://www.khronos.org/registry/SYCL/specs/sycl-2020/html/sycl-2020.html>
- [7] NVIDIA Corporation. 2022. NVIDIA H100 Tensor Core GPU Architecture. Whitepaper. NVIDIA Corporation, Santa Clara, CA, USA. <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c>
- [8] NVIDIA Corporation. 2023. CUDA C++ Programming Guide. NVIDIA Corporation, Santa Clara, CA, USA. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [9] NVIDIA Corporation. 2023. Parallel Thread Execution ISA Version 8.2. NVIDIA Corporation, Santa Clara, CA, USA. <https://docs.nvidia.com/cuda/parallel-thread-execution/>

- [10] Benjamin F. Spector, Simran Arora, Aaryan Singhal, Arjun Parthasarathy, Daniel Y. Fu, and Christopher Ré. 2025. ThunderKittens: Simple, Fast, and

Adorable AI Kernels. In *Proceedings of the 13th International Conference on Learning Representations (ICLR '25)*. OpenReview.net, Singapore, 24 pages. <https://openreview.net/forum?id=0fJfVOSUra>